Grid Maker

Project Document

Assignment:	Assignment 2 - Grid Maker
Start Date:	September 25, 2025
Due Date:	October 10, 2025, 11:59 PM
Project Duration:	2 weeks (16 days)
Team Size:	2 developers (David & Kevin)
Document Version:	2.0
Last Updated:	October 9, 2025

An interactive web application for creating and manipulating dynamic grids with color customization.

Contents

1 Feature Requirements **3**
1.1 Core Features . . . 3
1.1.1 Grid Structure Management . . . 3
1.1.2 Color Management System . . . 3
1.1.3 User Interface Elements . . . 3
1.2 Technical Requirements . . . 3
1.2.1 DOM Manipulation . . . 3
1.2.2 Event Handling . . . 3
1.2.3 Node Properties . . . 4
1.3 Version Control Requirements . . . 4
1.3.1 Git Workflow . . . 4
1.4 Deployment Requirements . . . 4

2 Application Architecture Description and Diagram **4**
2.1 Architecture Overview . . . 4
2.2 System Components . . . 4
2.2.1 Presentation Layer (HTML) . . . 4
2.2.2 Styling Layer (CSS) . . . 4
2.2.3 Application Logic Layer (JavaScript) . . . 5
2.3 Architecture Diagram . . . 5
2.4 Data Flow . . . 5
2.5 Technology Stack . . . 6
2.6 Key Design Patterns . . . 6

3 Epics, User Stories, and Acceptance Criteria **6**
3.1 Epic 1: Grid Structure Management . . . 6
3.1.1 User Story 1.1: Select Action from Menu . . . 6
3.1.2 User Story 1.2: Add Rows to Grid . . . 6
3.1.3 User Story 1.3: Add Columns to Grid . . . 7
3.1.4 User Story 1.4: Remove Rows from Grid . . . 7
3.1.5 User Story 1.5: Remove Columns from Grid . . . 7
3.2 Epic 2: Color Management System . . . 8
3.2.1 User Story 2.1: Select Color from Dropdown . . . 8
3.2.2 User Story 2.2: Color Single Cell . . . 8
3.2.3 User Story 2.3: Color All Uncolored Cells . . . 8
3.2.4 User Story 2.4: Color All Cells . . . 9
3.2.5 User Story 2.5: Clear All Cells . . . 9
3.3 Epic 3: Code Quality and Version Control . . . 9
3.3.1 User Story 3.1: Feature Branch Workflow . . . 9
3.3.2 User Story 3.2: Clean Code Organization . . . 10
3.4 Epic 4: Deployment and Documentation . . . 10
3.4.1 User Story 4.1: GitHub Pages Deployment . . . 10
3.4.2 User Story 4.2: Project Documentation . . . 10

4 Project Schedule Chart (Gantt Chart) **11**
4.1 Project Timeline Overview . . . 11
4.2 Detailed Gantt Chart . . . 11
4.3 Phase Breakdown . . . 11
4.3.1 Phase 1: Planning & Setup (Days 1-3, Sept 25-27) . . . 11

4.3.2 Phase 2: Grid Structure Management (Days 3-9, Sept 27-Oct 3) 12
4.3.3 Phase 3: Color Management System (Days 9-13, Oct 3-7) 13
4.3.4 Phase 4: Testing, Documentation & Deployment (Days 13-16, Oct 7-10) . 13
4.4 Milestones 14
4.5 Risk Management 14
4.5.1 Identified Risks 14
4.5.2 Mitigation Strategies 14
4.6 Resource Allocation 14
4.6.1 Team Structure (2-week timeline) 14
4.6.2 Recommended Task Division (4-Person Team) 15
4.6.3 Required Tools 15

5 Implementation Notes **15**
5.1 Recommended JavaScript Methods 15
5.1.1 Element Selection 15
5.1.2 DOM Manipulation 15
5.1.3 Node Properties 15
5.1.4 Event Handling 16
5.2 Color Suggestions 16
5.3 Best Practices 16

1 Feature Requirements

1.1 Core Features

1.1.1 Grid Structure Management

- **Add Rows**: Users can dynamically add rows to the grid
- **Add Columns**: Users can dynamically add columns to the grid
- **Remove Rows**: Users can remove rows from the existing grid
- **Remove Columns**: Users can remove columns from the existing grid
- **Dynamic Updates**: Grid updates immediately upon user actions

1.1.2 Color Management System

- **Color Selection**: Dropdown menu with multiple color options
- **Single Cell Coloring**: Click individual cells to apply the selected color
- **Fill Uncolored Cells**: Apply selected color to all uncolored cells at once
- **Fill All Cells**: Apply selected color to all cells regardless of current color
- **Clear Grid**: Reset all cells to their initial color (white)

1.1.3 User Interface Elements

- **Action Menu**: Dropdown menu for selecting grid operations
- **Color Picker**: Dropdown menu for selecting colors
- **Interactive Grid**: Clickable cells that respond to user interactions
- **Visual Feedback**: Clear indication of selected colors and cell states

1.2 Technical Requirements

1.2.1 DOM Manipulation

- Use of `getElementById()` for element selection
- Use of `getElementsByTagName()` for collection retrieval
- Use of `querySelector()` and `querySelectorAll()` for advanced selection
- Use of `createElement()` for dynamic element creation
- Use of `appendChild()` and `removeChild()` for DOM tree manipulation
- Use of `insertCell()`, `deleteCell()`, `insertRow()`, `deleteRow()` for table operations

1.2.2 Event Handling

- `onchange` events for dropdown menu selections
- `onclick` events for cell interactions
- Event listeners for user actions
- Dynamic event binding for newly created elements

1.2.3 Node Properties

- Use of `node.children` for accessing child elements
- Use of `node.lastElementChild` for accessing last child
- Traversal of DOM tree structure

1.3 Version Control Requirements

1.3.1 Git Workflow

- **Feature Branches**: Separate branch for each user story
- **Pull Requests**: Merge feature branches via pull requests
- **Commit Practices**: Small, frequent commits with descriptive messages
- **Branch Naming**: Clear, descriptive branch names (e.g., feature/add-rows)

1.4 Deployment Requirements

- Website deployed to GitHub Pages
- Live URL accessible and functional
- README includes link to deployed site
- All features working in production environment

2 Application Architecture Description and Diagram

2.1 Architecture Overview

The Grid Maker application follows a **client-side web application architecture** using vanilla JavaScript for DOM manipulation. The architecture emphasizes **event-driven programming** and **dynamic content generation**.

2.2 System Components

2.2.1 Presentation Layer (HTML)

- **User Interface**: Action menu, color picker, and grid container
- **Semantic Structure**: Proper HTML5 elements for accessibility
- **Dynamic Grid**: Table element that updates based on user actions
- **Form Controls**: Select elements for user input

2.2.2 Styling Layer (CSS)

- **Grid Styling**: Visual appearance of grid cells and borders
- **Control Styling**: Professional appearance for menus and buttons
- **Responsive Design**: Adaptable layout for different screen sizes
- **Color Themes**: Consistent color scheme throughout application
- **Hover Effects**: Visual feedback for interactive elements

2.2.3 Application Logic Layer (JavaScript)

- **Grid Manager**: Functions for adding/removing rows and columns
- **Color Manager**: Functions for applying colors to cells
- **Event Manager**: Handlers for user interactions
- **State Manager**: Tracking current selected action and color
- **Cell Manager**: Individual cell click handlers and color application

2.3 Architecture Diagram

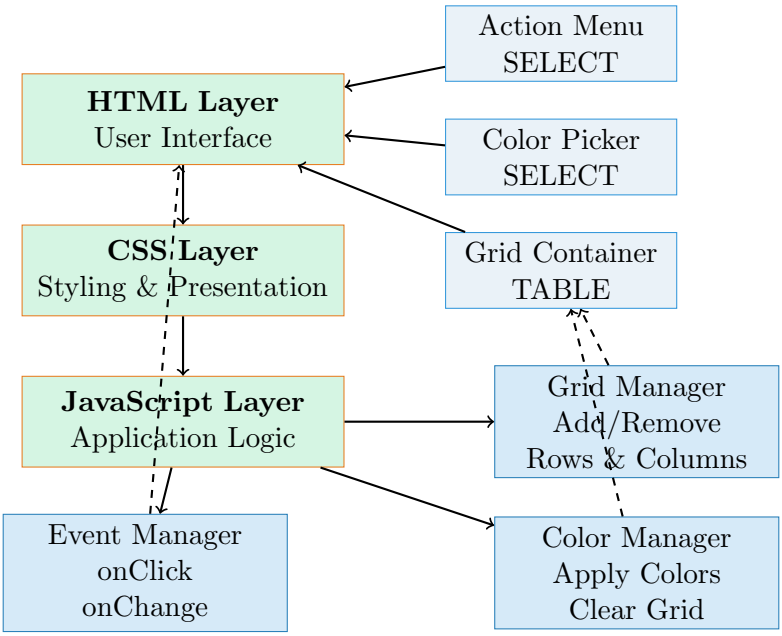


Figure 1: Grid Maker Application Architecture

2.4 Data Flow

1. **User Interaction**: User selects action from menu or color from picker
2. **Event Capture**: JavaScript event handlers capture the selection
3. **State Update**: Application updates internal state (selected action/color)
4. **Action Execution**: Based on selection, appropriate function is called
5. **DOM Manipulation**: Functions modify the DOM (add/remove cells, apply colors)
6. **Visual Update**: Browser re-renders the updated grid
7. **Event Binding**: New elements receive appropriate event listeners

2.5 Technology Stack

- **HTML5**: Structure and semantic markup
- **CSS3**: Styling, layout, and visual effects
- **JavaScript (ES6+)**: Application logic and DOM manipulation
- **Git**: Version control and collaboration
- **GitHub**: Repository hosting and collaboration
- **GitHub Pages**: Static website hosting

2.6 Key Design Patterns

- **Event-Driven Architecture**: User actions trigger event handlers
- **Separation of Concerns**: HTML (structure), CSS (presentation), JavaScript (behavior)
- **Modular Functions**: Each function has a single, well-defined responsibility
- **Progressive Enhancement**: Core functionality works, enhanced with JavaScript

3 Epics, User Stories, and Acceptance Criteria

3.1 Epic 1: Grid Structure Management

3.1.1 User Story 1.1: Select Action from Menu

As a user
I want to select an action from a dropdown menu
So that I can choose what operation to perform on the grid
Acceptance Criteria:

- Action menu is visible and accessible
- Menu includes all available actions (Add Row, Add Column, Remove Row, Remove Column, Fill Uncolored, Fill All, Clear Grid)
- Selected action is stored and used for subsequent operations
- Menu has clear, descriptive labels
- onChange event properly captures menu selection

3.1.2 User Story 1.2: Add Rows to Grid

As a user
I want to add rows to the grid
So that I can increase the vertical size of my grid
Acceptance Criteria:

- Selecting "Add Row" action adds a new row to the bottom of the grid
- New row contains the same number of cells as existing rows
- New cells have default white background color

- New cells are clickable and respond to color application
- Grid updates immediately without page refresh
- Uses `insertRow()` method

3.1.3 User Story 1.3: Add Columns to Grid

As a user
I want to add columns to the grid
So that I can increase the horizontal size of my grid
Acceptance Criteria:

- Selecting "Add Column" action adds a new column to the right side of all rows
- Each row receives one new cell at the end
- New cells have default white background color
- New cells are clickable and respond to color application
- Grid updates immediately without page refresh
- Uses `insertCell()` method

3.1.4 User Story 1.4: Remove Rows from Grid

As a user
I want to remove rows from the grid
So that I can decrease the vertical size of my grid
Acceptance Criteria:

- Selecting "Remove Row" action removes the last row from the grid
- Grid prevents removal if only one row remains (or implements minimum row logic)
- Grid updates immediately without page refresh
- No errors occur when attempting to remove from minimum grid
- Uses `deleteRow()` method

3.1.5 User Story 1.5: Remove Columns from Grid

As a user
I want to remove columns from the grid
So that I can decrease the horizontal size of my grid
Acceptance Criteria:

- Selecting "Remove Column" action removes the last cell from each row
- Grid prevents removal if only one column remains (or implements minimum column logic)
- Grid updates immediately without page refresh
- No errors occur when attempting to remove from minimum grid
- Uses `deleteCell()` method

3.2 Epic 2: Color Management System

3.2.1 User Story 2.1: Select Color from Dropdown

As a user
I want to select a color from a dropdown menu
So that I can choose which color to apply to grid cells
Acceptance Criteria:

- Color picker dropdown is visible and accessible
- Menu includes multiple color options (at least 5-7 colors)
- Color names are clear and descriptive
- Selected color is stored and used for cell coloring
- onChange event properly captures color selection
- Colors are standard HTML/CSS colors or hex values

3.2.2 User Story 2.2: Color Single Cell

As a user
I want to click on a cell to color it
So that I can color individual cells with my selected color
Acceptance Criteria:

- Clicking a cell applies the currently selected color to that cell
- Cell color updates immediately upon click
- Previously colored cells can be recolored
- onClick event properly captures cell clicks
- Cell background color changes visibly
- All cells are individually clickable

3.2.3 User Story 2.3: Color All Uncolored Cells

As a user
I want to fill all uncolored cells with the selected color
So that I can quickly color empty cells without clicking each one
Acceptance Criteria:

- Action applies selected color only to cells with default/white color
- Previously colored cells remain unchanged
- All uncolored cells update simultaneously
- Action works correctly regardless of grid size
- Uses `querySelectorAll()` or similar to find uncolored cells

3.2.4 User Story 2.4: Color All Cells

As a user
I want to fill all cells with the selected color
So that I can change the entire grid to a single color quickly
Acceptance Criteria:

- Action applies selected color to every cell in the grid
- All cells change color regardless of previous color
- All cells update simultaneously
- Action works correctly regardless of grid size
- Uses `getElementsByTagName()` or `querySelectorAll()` to select all cells

3.2.5 User Story 2.5: Clear All Cells

As a user
I want to clear all cell colors
So that I can reset the grid to its initial state
Acceptance Criteria:

- Action resets all cells to default white color
- All cells update simultaneously
- Grid structure remains unchanged (rows and columns preserved)
- Action works correctly regardless of grid size
- Clear action is easily accessible from action menu

3.3 Epic 3: Code Quality and Version Control

3.3.1 User Story 3.1: Feature Branch Workflow

As a developer
I want to create feature branches for each user story
So that I can develop features in isolation and merge them safely
Acceptance Criteria:

- Each user story has its own feature branch
- Branch names follow convention (e.g., feature/add-rows, feature/color-cells)
- Branches are created from main/master branch
- Feature branches are merged via pull requests
- Commit history shows clear feature development

3.3.2 User Story 3.2: Clean Code Organization

As a developer
I want to maintain clean, well-organized code
So that the code is easy to read, understand, and maintain
Acceptance Criteria:

- Code is properly indented and formatted
- Functions have descriptive names
- Variables have meaningful names
- Comments explain complex logic
- Code follows consistent style guidelines
- Separate files for HTML, CSS, and JavaScript

3.4 Epic 4: Deployment and Documentation

3.4.1 User Story 4.1: GitHub Pages Deployment

As a user
I want to access the Grid Maker online
So that I can use it without downloading files
Acceptance Criteria:

- Website is deployed to GitHub Pages
- Live URL is accessible and functional
- All features work in production environment
- README includes link to live site
- No console errors in production

3.4.2 User Story 4.2: Project Documentation

As a team member or external developer
I want to read clear documentation
So that I understand the project structure and how to contribute
Acceptance Criteria:

- README includes project description
- README lists group member names and GitHub usernames
- README includes link to GitHub Pages deployment
- README explains how to run the project locally
- Project document (PDF) includes all required sections

4 Project Schedule Chart (Gantt Chart)

4.1 Project Timeline Overview

Project Duration: 2 weeks (16 days)
Start Date: September 25, 2025
Due Date: October 10, 2025, 11:59 PM
Team Size: 1-4 developers

4.2 Detailed Gantt Chart

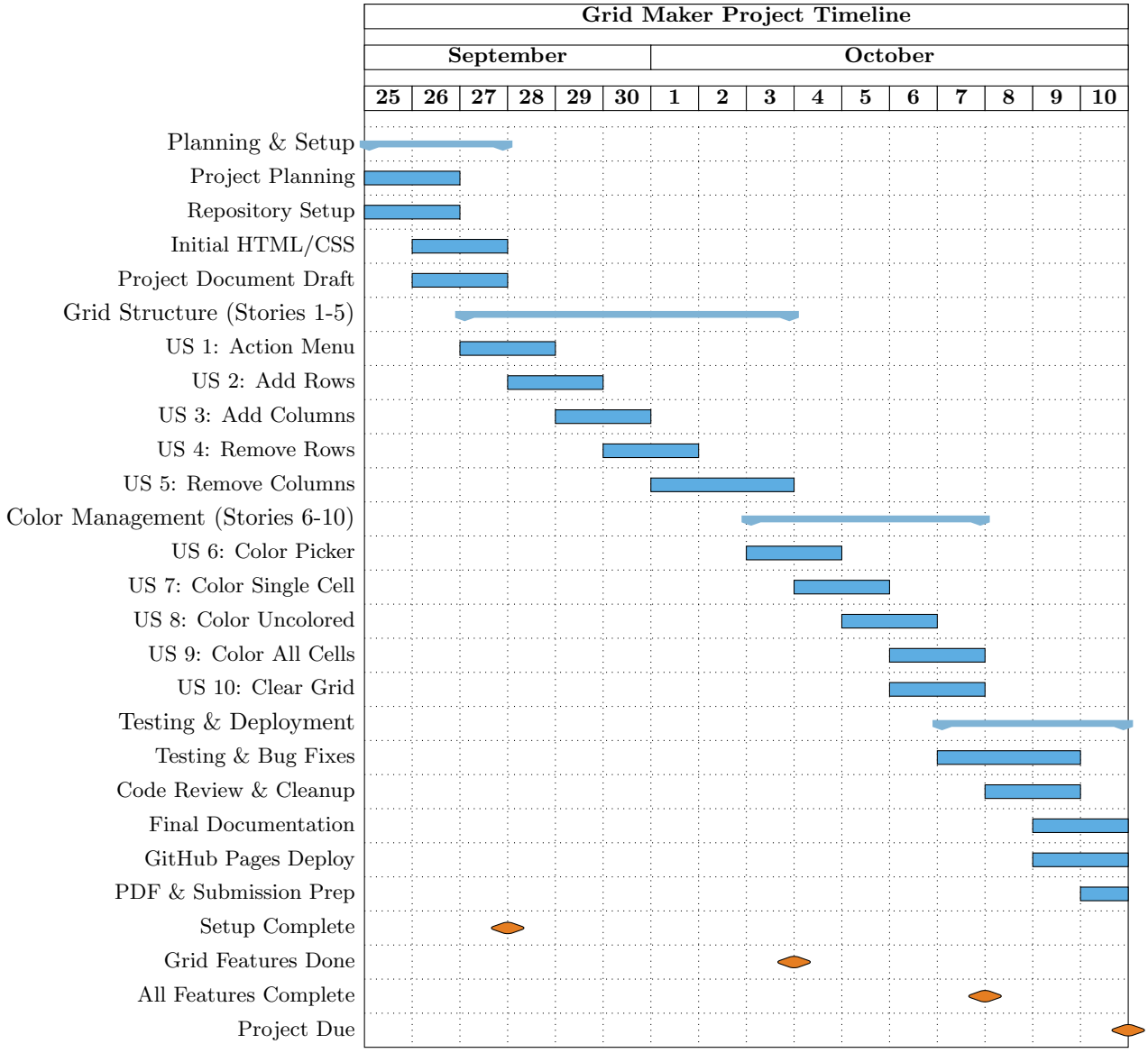


Figure 2: Project Timeline - Gantt Chart (Sept 25 - Oct 10, 2025)

4.3 Phase Breakdown

4.3.1 Phase 1: Planning & Setup (Days 1-3, Sept 25-27)

Tasks:

- Analyze project requirements
- Create project document structure
- Set up GitHub repository
- Create initial HTML structure with basic grid
- Set up CSS framework for styling
- Initialize JavaScript file
- Draft initial project documentation

Deliverables:

- Project document (initial draft)
- GitHub repository with README
- Basic HTML/CSS/JS file structure
- Initial grid display

Branch: `main/master` (initial setup)

4.3.2 Phase 2: Grid Structure Management (Days 3-9, Sept 27-Oct 3)

Tasks:

- Implement action menu dropdown
- Create add row functionality (feature/add-rows)
- Create add column functionality (feature/add-columns)
- Create remove row functionality (feature/remove-rows)
- Create remove column functionality (feature/remove-columns)
- Test grid manipulation across different sizes

Deliverables:

- Working action menu
- Complete row/column add/remove functionality
- 5 feature branches with pull requests
- Functional grid structure management

Branches:

- `feature/action-menu`
- `feature/add-rows`
- `feature/add-columns`
- `feature/remove-rows`
- `feature/remove-columns`

4.3.3 Phase 3: Color Management System (Days 9-13, Oct 3-7)

Tasks:

- Implement color picker dropdown
- Create single cell coloring functionality (feature/color-cell)
- Create fill uncolored cells functionality (feature/fill-uncolored)
- Create fill all cells functionality (feature/fill-all)
- Create clear grid functionality (feature/clear-grid)
- Test color application and clearing

Deliverables:

- Working color picker
- Complete color management system
- 5 feature branches with pull requests
- Fully functional grid coloring

Branches:

- `feature/color-picker`
- `feature/color-cell`
- `feature/fill-uncolored`
- `feature/fill-all`
- `feature/clear-grid`

4.3.4 Phase 4: Testing, Documentation & Deployment (Days 13-16, Oct 7-10)

Tasks:

- Comprehensive testing of all features
- Cross-browser testing
- Code cleanup and refactoring
- Add code comments
- Complete project document
- Update README with team info and deployment link
- Deploy to GitHub Pages
- Final testing of deployed site
- Prepare submission materials (PDF conversion)

Deliverables:

- Fully tested application

- Clean, commented code
- Complete project document (PDF)
- Updated README
- Live website on GitHub Pages
- Submission ready for Brightspace

4.4 Milestones

- **September 27**: Project setup and initial structure complete
- **October 3**: Core grid structure management complete
- **October 7**: All color management features complete
- **October 10**: Project deployed, tested, and submitted

4.5 Risk Management

4.5.1 Identified Risks

- **DOM Manipulation Complexity**: Table manipulation may have edge cases
- **Event Handler Issues**: Dynamically created cells need proper event binding
- **Browser Compatibility**: Different browsers may handle events differently
- **Merge Conflicts**: Multiple feature branches may conflict
- **Time Constraints**: One week is a tight timeline

4.5.2 Mitigation Strategies

- **Incremental Testing**: Test each feature thoroughly before moving to next
- **Event Delegation**: Use event delegation for dynamically created cells
- **Browser Testing**: Test in Chrome, Firefox, and Edge during development
- **Small Commits**: Make frequent, small commits to minimize conflicts
- **Early Start**: Begin project as soon as assigned
- **Code Reviews**: Review pull requests carefully before merging

4.6 Resource Allocation

4.6.1 Team Structure (2-week timeline)

- **Solo Developer**: All tasks (estimated 20-25 hours over 2 weeks)
- **2-Person Team**: Split features evenly (12-15 hours each over 2 weeks)
- **3-4 Person Team**: Divide by epics (10-12 hours each over 2 weeks)

4.6.2 Recommended Task Division (4-Person Team)

- **Developer 1**: Project setup, planning, add/remove rows features
- **Developer 2**: Action menu, add/remove columns features
- **Developer 3**: Color picker, single cell coloring, fill operations
- **Developer 4**: Clear grid, testing, documentation, deployment

4.6.3 Required Tools

- **Text Editor/IDE**: VS Code, Sublime Text, or similar
- **Web Browser**: Chrome, Firefox, or Edge with DevTools
- **Git**: For version control
- **GitHub Account**: For repository hosting and Pages
- **LaTeX Editor**: For project document (Overleaf, TeXShop, etc.)

5 Implementation Notes

5.1 Recommended JavaScript Methods

5.1.1 Element Selection

- `document.getElementById('gridTable')` - Get grid table element
- `document.getElementsByTagName('td')` - Get all cells
- `document.querySelector('#actionMenu')` - Get dropdown menus
- `document.querySelectorAll('td')` - Get all cells with selector

5.1.2 DOM Manipulation

- `table.insertRow()` or `table.insertRow(-1)` - Add row at end
- `table.deleteRow(-1)` - Remove last row
- `row.insertCell()` or `row.insertCell(-1)` - Add cell at end
- `row.deleteCell(-1)` - Remove last cell
- `document.createElement('td')` - Create new cell element
- `element.appendChild(child)` - Add child to element
- `element.removeChild(child)` - Remove child from element

5.1.3 Node Properties

- `table.children` or `table.rows` - Access rows
- `row.children` or `row.cells` - Access cells in row
- `table.lastElementChild` - Get last row
- `row.lastElementChild` - Get last cell in row

5.1.4 Event Handling

- `element.onchange = function(){...}` - Handle dropdown changes
- `element.onclick = function(){...}` - Handle click events
- `element.addEventListener('click', handler)` - Add event listener
- `event.target` - Get element that triggered event

5.2 Color Suggestions

Recommended colors for the color picker:

- White (`#FFFFFF`) - Default/reset color
- Red (`#FF0000` or `red`)
- Blue (`#0000FF` or `blue`)
- Green (`#00FF00` or `green`)
- Yellow (`#FFFF00` or `yellow`)
- Orange (`#FFA500` or `orange`)
- Purple (`#800080` or `purple`)
- Pink (`#FFC0CB` or `pink`)

5.3 Best Practices

- **Start Simple**: Begin with basic grid (e.g., 3x3) and build from there
- **Test Frequently**: Test each feature immediately after implementation
- **Use Console.log**: Debug issues by logging values to console
- **Event Delegation**: Consider using event delegation for cell clicks
- **Validate Input**: Check that rows/columns exist before removing
- **Consistent Naming**: Use clear, consistent variable and function names
- **Comment Complex Logic**: Explain non-obvious code sections
- **Commit Often**: Make small, frequent commits with clear messages